

INVENTORY MANAGEMENT SYSTEM SQL PROJECT REPORT

Introduction

The Inventory Management System project aims to analyze sales, customer behavior, and product performance using SQL.

The database consists of four main tables: Customer, Orders, OrderDetails, and Product, containing detailed information on customers, products, pricing, and order transactions.

By applying SQL queries, we can derive insights into sales trends, high-value customers, best-selling products, and overall business revenue.

This helps management make smarter decisions on stock control, pricing strategy, and marketing efforts.

Objectives

1. To analyze customer purchase patterns and spending behavior.
2. To calculate key business metrics such as total sales, order value, and revenue.
3. To identify best-selling and least-selling products.
4. To detect customers contributing the highest sales revenue.
5. To determine overall order trends over time.
6. To integrate product, order, and customer data using SQL joins for comprehensive insights.

-- =====

-- 1. Show all available products with their prices.

-- Purpose: To view the list of products currently available in inventory.

-- Which products are available in the system?

```
SELECT ProductID, ProductName, ProductListPrice FROM product;
```



ProductID	ProductName	ProductListPrice
P001	Intel Xeon E5-2699 V3 (OEM/Tray)	3410.46
P002	Intel Xeon E5-2697 V3	2774.98
P003	Intel Xeon E5-2698 V3 (OEM/Tray)	2660.72
P004	Intel Xeon E5-2697 V4	2554.99
P005	Intel Xeon E5-2685 V3 (OEM/Tray)	2501.69
P006	Intel Xeon E5-2695 V3 (OEM/Tray)	2431.95
P007	Intel Xeon E5-2697 V2	2377.09
P008	Intel Xeon E5-2695 V4	2269.99
P009	Intel Xeon E5-2695 V2	2259.99
P010	Intel Xeon E5-2643 V2 (OEM/Tray)	2200
P011	Intel Xeon E5-2690 (OEM/Tray)	2116.72

2. List all customers who have a credit limit above 4,000.

Purpose: To identify customers who can place higher-value orders.

-- =====

--

--

-- Which customers have a credit limit greater than 4,500?

```
SELECT CustomerName, CustomerCreditLimit FROM Customer WHERE CustomerCreditLimit > 4500;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
CustomerName	CustomerCreditLimit		
Basilia Downs	5000		
Sherron Flores	5000		
Cleo English	5000		
Ismael Solomon	5000		
Wendell Massey	5000		
Pearson Randall	5000		
Pham Hong	4900		
Purinton Janine	4900		

-- =====

-- 3. Find the total number of orders placed.

-- Purpose: To calculate total sales activity.

-- How many orders have been placed so far?

```
SELECT COUNT(*) FROM Orders;
```

Result Grid	Filter Rows:
COUNT(*)	
400	

-- =====

-- 4. Calculate the total revenue earned from all orders.

-- Purpose: To measure overall business revenue.

-- Calculate the total revenue earned from all orders.

```
SELECT SUM(OrderItemQuantity * PerUnitPrice) AS TotalRevenue FROM OrderDetails;
```

Result Grid	Filter Rows:
TotalRevenue	
30520090.710000005	

5. Find the average selling price of products.

Purpose: To understand product pricing trends.

-- =====
--
--

-- Find the average selling price of products.

```
SELECT round(AVG(ProductListPrice) ,2)AS AveragePrice FROM Product;
```

Result Grid	Filter Rows:
AveragePrice	
▶ 2002.4	

-- =====

-- 6. Which customer spent the most money overall?

-- Purpose: To find top-spending customers for rewards and retention.

```
SELECT o.CustomerID, SUM(od.OrderItemQuantity * od.PerUnitPrice) AS TotalSpent FROM Orders o JOIN OrderDetails od  
ON o.OrderID = od.OrderID GROUP BY o.CustomerID ORDER BY TotalSpent DESC LIMIT 1;
```

Result Grid	Filter Rows:
CustomerID TotalSpent	
▶ 2659 600155	

--

=====

-- 7. List customers who spent more than 3,00,000 in total.

-- Purpose: To identify high-value customers.

-- List customers who spent more than 3,00,000 in total.

```
SELECT o.CustomerID, SUM(od.OrderItemQuantity * od.PerUnitPrice) AS TotalSpent FROM Orders o JOIN OrderDetails od ON o.OrderID = od.OrderID  
GROUP BY o.CustomerID HAVING SUM(od.OrderItemQuantity * od.PerUnitPrice) > 300000;
```

Result Grid	Filter Rows:	Export:	Wra
CustomerID TotalSpent			
▶ 2401 323400			
2456 302398.88			
2513 300139.18			
2546 550487			
2567 328038.420000000004			
2573 344872.35			
2604 389855.7			
2607 307683.509999999995			
2659 600155			

8. Show the total quantity sold of each product.

Purpose: To determine best-selling products.

-- Show the total quantity sold of each product.

```
SELECT ProductID, SUM(OrderItemQuantity) AS TotalSold FROM OrderDetails GROUP BY ProductID;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
ProductID	TotalSold		
P001	132		
P002	124		
P003	92		
P004	128		
P005	106		
P006	148		
P007	127		
P008	138		
P009	117		
P010	34		

-- 9. Display top 3 products by total sales revenue.

-- Purpose: To identify most profitable products.

-- Display top 3 products by total sales revenue.

```
SELECT ProductID, SUM(OrderItemQuantity * PerUnitPrice) AS TotalRevenue FROM OrderDetails GROUP BY ProductID ORDER BY TotalRevenue DESC LIMIT 3;
```

Result Grid	Filter Rows:	Export:
ProductID	TotalRevenue	
P359	600155	
P246	550487	
P304	389855.7	

-- 10. Find products that were never ordered.

-- Purpose: To identify products that are in stock but unsold.

-- Find products that were never ordered.

```
SELECT ProductName FROM Product WHERE ProductID NOT IN (SELECT ProductID FROM OrderDetails);
```

Result Grid	Filter Rows:	Export:
ProductName		

11. Find total value of each order.

Purpose: To compare large and small orders.

-- =====

--

--

-- Find total value of each order.

```
SELECT OrderID, SUM(OrderItemQuantity * PerUnitPrice) AS OrderValue FROM OrderDetails GROUP BY OrderID;
```

Result Grid	Filter Rows:	Export:
OrderID	OrderValue	
7272	62038.68	
7273	64478.76	
7274	73668.08	
7275	108798.72	
7276	11658.939999999999	
7277	81339.32	
7278	83818.73	
7279	7588.62	
7280	93396.42	

-- =====

-- 12. Show orders placed in the year 2013.

-- Purpose: To analyze order trends.

-- Show orders placed in the year 2013.

```
SELECT * FROM Orders WHERE YEAR(OrderDate) = 2013;
```

Result Grid	Filter Rows:	Export:
OrderID	OrderDate	CustomerID
7347	2013/06/21	2376
7413	2013/06/21	2442
7496	2013/06/21	2525
7571	2013/06/21	2600
7626	2013/06/21	2655

-- =====

-- 13. Display each customer's name with their order IDs.

-- Purpose: To link customer data with their corresponding orders.

-- Display each customer's name with their order IDs.

```
SELECT c.CustomerName, o.OrderID FROM Customer c JOIN Orders o ON c.CustomerID = o.CustomerID;
```

Result Grid			Filter Rows:	Export:	Wrap
	CustomerName	OrderID			
▶	Flor Stone	7272			
	Lavera Emerson	7273			
	Fern Head	7274			
	Shyla Ortiz	7275			
	Jeni Levy	7276			
	Matthias Hannah	7277			
	Matthias Cruise	7278			
	Meenakshi Mason	7279			
	Christian Cage	7280			

-- =====

-- 14. Show product names in each order.

-- Purpose: To check which products were included in which orders.

-- Show product names in each order.

```
SELECT o.OrderID, p.ProductName FROM Orders o JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Product p ON od.ProductID = p.ProductID;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	OrderID	ProductName			
▶	7272	Intel Xeon E5-2699 V3 (OEM/Tray)			
	7273	Intel Xeon E5-2697 V3			
	7274	Intel Xeon E5-2698 V3 (OEM/Tray)			
	7275	Intel Xeon E5-2697 V4			
	7276	Intel Xeon E5-2685 V3 (OEM/Tray)			
	7277	Intel Xeon E5-2695 V3 (OEM/Tray)			
	7278	Intel Xeon E5-2697 V2			
	7279	Intel Xeon E5-2695 V4			
	7280	Intel Xeon E5-2695 V2			
	7281	Intel Xeon E5-2695 V3 (OEM/Tray)			

-- =====

-- 15. Find which customer ordered which product and total amount spent per product.

-- Purpose: To combine sales, product, and customer data for deeper insights.

-- Find which customer ordered which product and total amount spent per product.

```
SELECT c.CustomerName, p.ProductName, SUM(od.OrderItemQuantity * od.PerUnitPrice) AS AmountSpent
FROM Customer c JOIN Orders o ON c.CustomerID = o.CustomerID JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Product p ON od.ProductID = p.ProductID GROUP BY c.CustomerName, p.ProductName;
```

Result Grid

  Filter Rows:

Export: 

Wrap Cell Content: 

	CustomerName	ProductName	AmountSpent
▶	Flor Stone	Intel Xeon E5-2699 V3 (OEM/Tray)	62038.68
	Lavera Emerson	Intel Xeon E5-2697 V3	64478.76
	Fern Head	Intel Xeon E5-2698 V3 (OEM/Tray)	73668.08
	Shyla Ortiz	Intel Xeon E5-2697 V4	108798.72
	Jeni Levy	Intel Xeon E5-2685 V3 (OEM/Tray)	11658.939999999999
	Matthias Hannah	Intel Xeon E5-2695 V3 (OEM/Tray)	81339.32
	Matthias Cruise	Intel Xeon E5-2697 V2	83818.73
	Meenakshi Mason	Intel Xeon E5-2695 V4	7588.62
	Christian Cage	Intel Xeon E5-2695 V2	93396.42
	Charlie Sutherland	Intel Xeon E5-2643 V2 (OEM/Tray)	9519.66
	Charlie Pacino	Intel Xeon E5-2690 (OEM/Tray)	31876.22

Key Metrics Summary

Metric	SQL Query Reference	Business Insight
Total Orders	400	Shows overall order volume.
Total Revenue	30520090.71	Measures total income from all orders.
Average Product Price	2022.4	Helps understand pricing strategy.
Top Customer	Shields Seffi	Identifies the highest spender.
High-Value Customers (>3,00,000)	2401,2456,2531, 2546,2547....	Highlights premium or loyal customers.
Top 3 Products (Revenue)	PNY SSD9SC240GMDA-RB, Samsung MZV6E500, Kingston Fury (HX316C10FB/8) 8 GB	Reveals best-performing items.
Unsold Products	none	Points out products not generating sales.
Customer–Order Mapping	Many customers with multiple orders	Shows customer engagement and repeat order frequency.
Customer–Product Spending	Detailed mapping of customers vs. items purchased	Reveals purchase patterns and individual product demand.

Conclusion

The Inventory Management System SQL Project successfully demonstrates how SQL can be applied to extract valuable insights from sales and inventory data.

Key findings include:

- Identification of high-value customers (spending above ₹3,00,000).
- Recognition of top-performing products driving major revenue.
- Detection of unsold items and areas of improvement in stock management.
- Observation of sales activity during 2013, showing time-based order trends.

Overall, this project proves that SQL is not only a data tool but a decision-making asset — helping businesses optimize sales performance, improve stock efficiency, and strengthen customer relationships.